CODING INTERVIEW PREPARATION

Complete Guide to Crack Coding Interviews

RESOURCE	DETAIL
Questions	100 coding & CS interview questions
Coverage	DSA • Coding Patterns • OOP • DB • API • System Design
Languages	JavaScript examples + language-independent concepts
Difficulty	Beginner → Intermediate → Interview level
Format	Colorful, structured, print-friendly PDF

How to use this guide: First understand the pattern, then write the solution yourself, then compare complexity and edge cases. Do not memorize code without understanding why it works.

Recommended interview flow: Clarify → give examples → explain brute force → optimize → state time/space complexity → code → test edge cases.

CODING PATTERN CHEAT SHEET

Pattern	When to think about it	Typical clue
Hash Map / Set	Fast lookup, duplicates, complements	Two Sum, frequency
Two Pointers	Sorted array or opposite ends	Pair sum, palindrome
Sliding Window	Contiguous subarray/substring	Longest/shortest window
Stack	Nested structure / nearest greater	Parentheses, monotonic stack
Queue / BFS	Level order / shortest unweighted path	Tree levels, graph distance
Binary Search	Sorted or monotonic search space	First/last valid, target
DFS	Explore components/paths deeply	Trees, graphs, backtracking
Heap	Repeated min/max selection	Kth largest, priority queue
Prefix Sum	Many range-sum queries	Subarray sums
Greedy	Local choice can be proven optimal	Intervals, scheduling
DP	Overlapping subproblems + optimal substructure	Knapsack, stairs
Backtracking	Enumerate valid combinations	Subsets, permutations

Complexity targets: Prefer O(n) over O(n^2) when possible; O(log n) is excellent for ordered search; for output-heavy problems, remember that producing the output can itself require substantial time.

DATA STRUCTURES

Arrays & Strings

Q1. Find the largest element in an array.

Solution: Scan the array once while maintaining a maximum value. Initialize max to the first element and update it whenever a larger value appears.

Complexity: O(n) time, O(1) extra space

```
function maxElement(arr) {
  let mx = arr[0];
  for (const x of arr) mx = Math.max(mx, x);
  return mx;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q2. Find the second-largest distinct element.

Solution: Maintain the largest and second-largest distinct values in one pass. Update both when a new maximum is found.

Complexity: O(n) time, O(1) extra space

```
function secondLargest(arr) {
  let first = -Infinity, second = -Infinity;
  for (const x of arr) {
    if (x > first) { second = first; first = x; }
    else if (x > second && x !== first) second = x;
  }
  return second;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q3. Reverse an array in-place.

Solution: Use two pointers from the beginning and end and swap until they meet.

Complexity: O(n) time, O(1) extra space

```
function reverseArray(a) {
  let l = 0, r = a.length - 1;
  while (l < r) [a[l], a[r]] = [a[r], a[l]], l++, r--;
  return a;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q4. Move all zeroes to the end while preserving order.

Solution: Use a write pointer for the next non-zero position. Copy non-zero values forward, then fill the remaining positions with zero.

Complexity: O(n) time, O(1) extra space

```
function moveZeroes(a) {
  let w = 0;
  for (const x of a) if (x !== 0) a[w++] = x;
  while (w < a.length) a[w++] = 0;
  return a;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q5. Remove duplicates from a sorted array.

Solution: Use a slow pointer. When the current value differs from the last retained value, write it at the next slow position.

Complexity: O(n) time, O(1) extra space

```
function removeDuplicates(a) {
  if (!a.length) return 0;
  let w = 1;
```

```
  for (let i = 1; i < a.length; i++)
    if (a[i] !== a[i-1]) a[w++] = a[i];
  return w;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q6. Solve Two Sum.

Solution: Store previously seen values in a hash map. For each value x, check whether target-x has already been seen.

Complexity: O(n) average time, O(n) space

```
function twoSum(nums, target) {
  const map = new Map();
  for (let i = 0; i < nums.length; i++) {
    const need = target - nums[i];
    if (map.has(need)) return [map.get(need), i];
    map.set(nums[i], i);
  }
  return [];
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q7. Find the maximum subarray sum.

Solution: Use Kadane's algorithm. At each position, decide whether to extend the current subarray or start a new one.

Complexity: O(n) time, O(1) space

```
function maxSubArray(a) {
  let cur = a[0], best = a[0];
  for (let i = 1; i < a.length; i++) {
    cur = Math.max(a[i], cur + a[i]);
    best = Math.max(best, cur);
  }
  return best;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q8. Rotate an array to the right by k positions.

Solution: Use the reversal method: reverse the whole array, reverse the first k elements, then reverse the remaining elements.

Complexity: O(n) time, O(1) extra space

```
function rotate(a, k) {
  k %= a.length;
  const rev = (l, r) => {
    while (l < r) [a[l], a[r]] = [a[r], a[l]], l++, r--;
  };
  rev(0, a.length-1); rev(0, k-1); rev(k, a.length-1);
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q9. Merge two sorted arrays.

Solution: Use two pointers and repeatedly append the smaller current element. Append the remaining tail afterward.

Complexity: O(n+m) time, O(n+m) output space

```
function merge(a, b) {
  let i=0,j=0,res=[];
  while(i<a.length && j<b.length)
    res.push(a[i] <= b[j] ? a[i++] : b[j++]);
  return res.concat(a.slice(i), b.slice(j));
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q10. Find the missing number from 0..n.

Solution: Use XOR. XORing all indices and all values cancels equal numbers and leaves the missing one.

Complexity: O(n) time, O(1) space

```
function missingNumber(a) {
  let x = a.length;
  for (let i=0;i<a.length;i++) x ^= i ^ a[i];
  return x;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q11. Check whether two strings are anagrams.

Solution: Count characters in one string and decrement using the second string. All counts must finish at zero.

Complexity: O(n) time, O(k) space

```
function isAnagram(a,b) {
  if (a.length !== b.length) return false;
  const m = new Map();
  for (const c of a) m.set(c,(m.get(c)||0)+1);
  for (const c of b) {
    if (!m.has(c)) return false;
    m.set(c,m.get(c)-1);
    if (m.get(c) < 0) return false;
  }
  return true;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q12. Find the first non-repeating character.

Solution: Count all characters first, then scan again and return the first character with count 1.

Complexity: O(n) time, O(k) space

```
function firstUnique(s) {
  const c = new Map();
  for (const ch of s) c.set(ch,(c.get(ch)||0)+1);
  for (const ch of s) if (c.get(ch) === 1) return ch;
  return null;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q13. Reverse the words in a sentence.

Solution: Split on whitespace, reverse the word list, and join with single spaces.

Complexity: O(n) time, O(n) space

```
function reverseWords(s) {
  return s.trim().split(/\s+/).reverse().join(" ");
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q14. Check whether a string is a palindrome.

Solution: Compare characters from both ends using two pointers.

Complexity: O(n) time, O(1) extra space

```
function isPalindrome(s) {
  let l=0,r=s.length-1;
  while(l<r) if(s[l++] !== s[r--]) return false;
  return true;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q15. Find the longest substring without repeating characters.

Solution: Use a sliding window and store the last index of each character. Move the left boundary when a repeated character enters the window.

Complexity: O(n) time, O(k) space

```
function longestUnique(s) {
  const last = new Map(); let l=0, best=0;
  for (let r=0;r<s.length;r++) {
    if (last.has(s[r])) l=Math.max(l,last.get(s[r])+1);
```

```
      last.set(s[r],r);
      best=Math.max(best,r-l+1);
    }
    return best;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q16. Find the majority element.

Solution: Use Boyer-Moore voting. A majority element cancels out with non-majority elements and still remains as the candidate.

Complexity: O(n) time, O(1) space

```
function majority(a) {
  let cand=null,count=0;
  for (const x of a) {
    if (count===0) cand=x;
    count += x===cand ? 1 : -1;
  }
  return cand;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q17. Find an intersection of two arrays.

Solution: Store values of the first array in a Set, then scan the second array and collect values present in the set.

Complexity: O(n+m) average time, O(n) space

```
function intersection(a,b) {
  const s=new Set(a), out=new Set();
  for(const x of b) if(s.has(x)) out.add(x);
  return [...out];
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q18. Find the product of array elements except self.

Solution: Build prefix products from the left, then multiply by suffix products from the right. No division is required.

Complexity: O(n) time, O(1) extra space excluding output

```
function productExceptSelf(a) {
  const out=Array(a.length).fill(1);
  let p=1;
  for(let i=0;i<a.length;i++){ out[i]=p; p*=a[i]; }
  p=1;
  for(let i=a.length-1;i>=0;i--){ out[i]*=p; p*=a[i]; }
  return out;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q19. Find the best time to buy and sell a stock once.

Solution: Track the minimum price seen so far and the best profit obtained by selling today.

Complexity: O(n) time, O(1) space

```
function maxProfit(p) {
  let min=Infinity, best=0;
  for(const x of p){ min=Math.min(min,x); best=Math.max(best,x-min); }
  return best;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Q20. Find all duplicate values in an array containing values 1..n.

Solution: Use the array itself as a marker structure: for value x, inspect index abs(x)-1 and negate it when first seen.

Complexity: O(n) time, O(1) extra space

```
function duplicates(a) {
  const out=[];
  for(const x of a){
```

```
    const i=Math.abs(x)-1;
    if(a[i]<0) out.push(Math.abs(x));
    else a[i]=-a[i];
  }
  return out;
}
```

Interview Tip: Always state edge cases such as empty input, one element, duplicates, negative values and integer overflow.

Linked Lists

Q21. Reverse a singly linked list.

Solution: Keep prev, current and next pointers; reverse current.next to prev and advance.

Complexity: O(n) time, O(1) space

```
function reverse(head){
  let prev=null, cur=head;
  while(cur){ const next=cur.next; cur.next=prev; prev=cur; cur=next; }
  return prev;
}
```

Interview Tip: Draw the pointers on paper before coding. Pointer mistakes are easier to catch visually.

Q22. Find the middle of a linked list.

Solution: Use slow and fast pointers. Slow moves one step while fast moves two steps.

Complexity: O(n) time, O(1) space

```
function middle(head){
  let slow=head, fast=head;
  while(fast && fast.next){ slow=slow.next; fast=fast.next.next; }
  return slow;
}
```

Interview Tip: Draw the pointers on paper before coding. Pointer mistakes are easier to catch visually.

Q23. Detect a cycle in a linked list.

Solution: Use Floyd's algorithm. If slow and fast pointers meet, a cycle exists.

Complexity: O(n) time, O(1) space

```
function hasCycle(head){
  let s=head,f=head;
  while(f && f.next){ s=s.next; f=f.next.next; if(s===f) return true; }
  return false;
}
```

Interview Tip: Draw the pointers on paper before coding. Pointer mistakes are easier to catch visually.

Q24. Merge two sorted linked lists.

Solution: Compare the current nodes and attach the smaller one to a dummy result node.

Complexity: O(n+m) time, O(1) extra space

```
function mergeLists(a,b){
  const dummy={next:null}; let t=dummy;
  while(a&&b){ if(a.val<=b.val){t.next=a;a=a.next;} else {t.next=b;b=b.next;} t=t.next; }
  t.next=a||b; return dummy.next;
}
```

Interview Tip: Draw the pointers on paper before coding. Pointer mistakes are easier to catch visually.

Q25. Remove the nth node from the end.

Solution: Use two pointers separated by n nodes, then move both until the fast pointer reaches the end.

Complexity: O(n) time, O(1) space

```
function removeNth(head,n){
  const d={next:head}; let a=d,b=d;
  for(let i=0;i<n;i++) b=b.next;
  while(b.next){a=a.next;b=b.next;}
  a.next=a.next.next; return d.next;
}
```

Interview Tip: Draw the pointers on paper before coding. Pointer mistakes are easier to catch visually.

Q26. Check whether a linked list is a palindrome.

Solution: Find the midpoint, reverse the second half, then compare both halves.

Complexity: O(n) time, O(1) extra space

```
// Find middle -> reverse second half -> compare corresponding nodes.
function isPal(head){
  let s=head,f=head;
  while(f&&f.next){s=s.next;f=f.next.next;}
  let prev=null;
  while(s){let n=s.next;s.next=prev;prev=s;s=n;}
  for(let a=head,b=prev;b;b=b.next,a=a.next) if(a.val!==b.val) return false;
  return true;
}
```

Interview Tip: Draw the pointers on paper before coding. Pointer mistakes are easier to catch visually.

Q27. Find the intersection node of two linked lists.

Solution: Use two pointers that switch to the other list at the end. They traverse equal combined lengths and meet at the intersection.

Complexity: O(n+m) time, O(1) space

```
function intersection(a,b){
  let p=a,q=b;
  while(p!==q){ p=p?p.next:b; q=q?q.next:a; }
  return p;
}
```

Interview Tip: Draw the pointers on paper before coding. Pointer mistakes are easier to catch visually.

Q28. Delete a node when only that node is given.

Solution: Copy the next node's value into the current node and bypass the next node. This cannot be used for the tail node.

Complexity: O(1) time, O(1) space

```
function deleteNode(node){
  node.val=node.next.val;
  node.next=node.next.next;
}
```

Interview Tip: Draw the pointers on paper before coding. Pointer mistakes are easier to catch visually.

Stacks & Queues

Q29. Implement a stack using an array.

Solution: Use push for insertion and pop for removal. The last inserted item is removed first.

Complexity: O(1) average per operation

```
class Stack{
  constructor(){this.a=[];}
  push(x){this.a.push(x);}
  pop(){return this.a.pop();}
  peek(){return this.a[this.a.length-1];}
}
```

Interview Tip: For stack/queue problems, identify whether the required order is LIFO or FIFO before selecting the data structure.

Q30. Implement a queue using two stacks.

Solution: Use one stack for incoming elements and one for outgoing elements. Transfer only when the outgoing stack is empty.

Complexity: Amortized O(1) per operation

```
class Queue{
  constructor(){this.in=[];this.out=[];}
  push(x){this.in.push(x);}
  pop(){
    if(!this.out.length) while(this.in.length) this.out.push(this.in.pop());
```

```
    return this.out.pop();
  }
}
```

Interview Tip: For stack/queue problems, identify whether the required order is LIFO or FIFO before selecting the data structure.

Q31. Check balanced parentheses.

Solution: Push opening brackets. For a closing bracket, the top must contain the matching opening bracket.

Complexity: O(n) time, O(n) space

```
function valid(s){
  const st=[], pair={')':'(',']':'[','}':'{'};
  for(const c of s){
    if('([{'.includes(c)) st.push(c);
    else if(st.pop()!==pair[c]) return false;
  }
  return st.length===0;
}
```

Interview Tip: For stack/queue problems, identify whether the required order is LIFO or FIFO before selecting the data structure.

Q32. Find the next greater element.

Solution: Scan from right to left with a monotonic decreasing stack. Remove values that cannot be the next greater candidate.

Complexity: O(n) time, O(n) space

```
function nextGreater(a){
  const st=[],res=Array(a.length).fill(-1);
  for(let i=a.length-1;i>=0;i--){
    while(st.length&&st[st.length-1]<=a[i]) st.pop();
    if(st.length) res[i]=st[st.length-1];
    st.push(a[i]);
  }
  return res;
}
```

Interview Tip: For stack/queue problems, identify whether the required order is LIFO or FIFO before selecting the data structure.

Q33. Evaluate a postfix expression.

Solution: Use a stack. Push operands; when an operator appears, pop the required operands, apply the operation and push the result.

Complexity: O(n) time, O(n) space

```
// Example tokens: ["2","3","+","4","*"] -> 20
function evalPostfix(t){
  const s=[];
  for(const x of t){
    if(!isNaN(x)) s.push(Number(x));
    else { const b=s.pop(),a=s.pop(); s.push(eval(`${a}${x}${b}`)); }
  }
  return s.pop();
}
```

Interview Tip: For stack/queue problems, identify whether the required order is LIFO or FIFO before selecting the data structure.

Q34. Design a circular queue.

Solution: Maintain an array, front index, rear index and size. Use modulo arithmetic so indices wrap around.

Complexity: O(1) per operation

```
// Core rule:
rear = (rear + 1) % capacity;
front = (front + 1) % capacity;
```

Interview Tip: For stack/queue problems, identify whether the required order is LIFO or FIFO before selecting the data structure.

Trees, BSTs & Heaps

Q35. What is tree traversal? Explain inorder, preorder and postorder.

Solution: Inorder visits left-root-right, preorder root-left-right, and postorder left-right-root. For a BST, inorder traversal produces sorted order.

Complexity: O(n) time, O(h) recursion stack

```
function inorder(root,out=[]){
  if(!root)return out;
  inorder(root.left,out); out.push(root.val); inorder(root.right,out);
  return out;
}
```

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

Q36. Find the height of a binary tree.

Solution: The height is one plus the maximum height of its left and right subtrees. Define the empty tree height consistently.

Complexity: O(n) time, O(h) space

```
function height(root){
  if(!root) return 0;
  return 1 + Math.max(height(root.left),height(root.right));
}
```

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

Q37. Find the maximum depth of a binary tree.

Solution: Maximum depth is the longest root-to-leaf path measured in nodes (or edges if that convention is specified).

Complexity: O(n) time, O(h) space

```
function maxDepth(root){
  if(!root)return 0;
  return 1+Math.max(maxDepth(root.left),maxDepth(root.right));
}
```

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

Q38. Level-order traversal of a binary tree.

Solution: Use a queue and process nodes level by level.

Complexity: O(n) time, O(w) space

```
function levelOrder(root){
  if(!root)return [];
  const q=[root],res=[];
  for(let i=0;i<q.length;i++){
    const x=q[i]; res.push(x.val);
    if(x.left)q.push(x.left); if(x.right)q.push(x.right);
  }
  return res;
}
```

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

Q39. Search for a value in a BST.

Solution: At each node, compare the target with the node value and move left or right accordingly.

Complexity: O(h) average; O(n) worst-case

```
function search(root,x){
  while(root){
    if(root.val===x)return root;
    root=x<root.val?root.left:root.right;
  }
  return null;
}
```

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

Q40. Validate whether a binary tree is a BST.

Solution: Carry lower and upper bounds down the tree. Each node must fall strictly within the valid range under the chosen duplicate policy.

Complexity: O(n) time, O(h) space

```
function validBST(root,lo=-Infinity,hi=Infinity){
  if(!root)return true;
  if(root.val<=lo||root.val>=hi)return false;
  return validBST(root.left,lo,root.val)&&validBST(root.right,root.val,hi);
}
```

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

Q41. Find the lowest common ancestor in a BST.

Solution: If both targets are smaller, go left; if both are larger, go right. Otherwise the current node is the split point.

Complexity: O(h) time, O(1) iterative space

```
function lca(root,a,b){
  while(root){
    if(a<root.val&&b<root.val) root=root.left;
    else if(a>root.val&&b>root.val) root=root.right;
    else return root;
  }
}
```

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

Q42. What is a balanced tree?

Solution: A balanced tree keeps subtree heights reasonably close so operations do not degrade badly. AVL trees enforce a strict height-balance condition; other balanced trees use different rules.

Complexity: Typically O(log n) search/update

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

Q43. What is a heap?

Solution: A heap is a complete binary tree satisfying the heap property. In a min-heap the smallest element is at the root; in a max-heap the largest is at the root.

Complexity: Peek O(1); insert/delete O(log n)

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

Q44. How does a priority queue work?

Solution: A priority queue removes the element with highest or lowest priority rather than the earliest inserted element. A binary heap is a common implementation.

Complexity: Heap insert/delete O(log n)

Interview Tip: Always ask whether the interviewer defines height in nodes or edges; both conventions exist.

ALGORITHMS

Graphs

Q45. What is BFS and when is it useful?

Solution: Breadth-first search explores vertices level by level using a queue. In an unweighted graph, BFS can find shortest paths by number of edges.

Complexity: O(V+E) time

Interview Tip: Clarify whether the graph is directed/undirected and weighted/unweighted before choosing an algorithm.

Q46. What is DFS?

Solution: Depth-first search explores as far as possible along a branch before backtracking. It can be implemented recursively or with an explicit stack.

Complexity: O(V+E) time

Interview Tip: Clarify whether the graph is directed/undirected and weighted/unweighted before choosing an algorithm.

Q47. Detect a cycle in an undirected graph.

Solution: Run DFS/BFS while tracking the parent. Encountering an already visited neighbor that is not the parent indicates a cycle.

Complexity: O(V+E) time

Interview Tip: Clarify whether the graph is directed/undirected and weighted/unweighted before choosing an algorithm.

Q48. Find shortest path in an unweighted graph.

Solution: Run BFS from the source and record distance when a vertex is first discovered.

Complexity: O(V+E) time

Interview Tip: Clarify whether the graph is directed/undirected and weighted/unweighted before choosing an algorithm.

Q49. What is Dijkstra's algorithm?

Solution: Dijkstra finds shortest paths from one source when edge weights are non-negative. It repeatedly extracts the closest unsettled vertex and relaxes outgoing edges.

Complexity: O((V+E) log V) with a binary heap

Interview Tip: Clarify whether the graph is directed/undirected and weighted/unweighted before choosing an algorithm.

Q50. What is topological sorting?

Solution: It orders vertices of a directed acyclic graph so every directed edge u→v places u before v. Kahn's algorithm uses indegrees; DFS is another approach.

Complexity: O(V+E) time

Interview Tip: Clarify whether the graph is directed/undirected and weighted/unweighted before choosing an algorithm.

Q51. What is a spanning tree?

Solution: A spanning tree connects all vertices of a connected undirected graph with no cycles and exactly V−1 edges.

Complexity: Construction commonly O(V+E)

Interview Tip: Clarify whether the graph is directed/undirected and weighted/unweighted before choosing an algorithm.

Q52. BFS vs DFS for shortest path?

Solution: For unweighted graphs, BFS guarantees the minimum number of edges. DFS does not generally guarantee a shortest path.

Complexity: BFS O(V+E)

Interview Tip: Clarify whether the graph is directed/undirected and weighted/unweighted before choosing an algorithm.

Searching & Sorting

Q53. Explain linear search.

Solution: Check each element sequentially until the target is found or the array ends.

Complexity: O(n) time, O(1) space

```
function linearSearch(a,x){
  for(let i=0;i<a.length;i++) if(a[i]===x) return i;
  return -1;
}
```

Interview Tip: Do not quote complexity without checking the exact variant and implementation.

Q54. Explain binary search.

Solution: On a sorted array, compare the target with the middle element and discard half of the search range each iteration.

Complexity: O(log n) time, O(1) space

```
function binarySearch(a,x){
  let l=0,r=a.length-1;
  while(l<=r){
    const m=l+Math.floor((r-l)/2);
    if(a[m]===x)return m;
    if(a[m]<x)l=m+1; else r=m-1;
  }
  return -1;
}
```

Interview Tip: Do not quote complexity without checking the exact variant and implementation.

Q55. Explain bubble sort.

Solution: Repeatedly swap adjacent elements that are out of order. After each pass, the largest remaining element moves to the end.

Complexity: $O(n^2)$ worst/average; O(n) best with optimization

Interview Tip: Do not quote complexity without checking the exact variant and implementation.

Q56. Explain insertion sort.

Solution: Build a sorted prefix by inserting each new element into its correct position within the prefix.

Complexity: $O(n^2)$ worst; O(n) best

Interview Tip: Do not quote complexity without checking the exact variant and implementation.

Q57. Explain merge sort.

Solution: Divide the array into halves, recursively sort each half, then merge the sorted halves.

Complexity: O(n log n) time, O(n) extra space

Interview Tip: Do not quote complexity without checking the exact variant and implementation.

Q58. Explain quicksort.

Solution: Choose a pivot, partition elements around it, then recursively sort both partitions.

Complexity: Average O(n log n), worst $O(n^2)$; stack O(log n) average

Interview Tip: Do not quote complexity without checking the exact variant and implementation.

Q59. Which sorting algorithm is stable?

Solution: A stable sort preserves the relative order of equal keys. Merge sort and insertion sort are commonly stable; standard in-place quicksort is generally not stable.

Complexity: Depends on algorithm/implementation

Interview Tip: Do not quote complexity without checking the exact variant and implementation.

Q60. When should you use binary search?

Solution: Use it when the search space is ordered or when a monotonic predicate lets you eliminate half the candidates at each step.

Complexity: Typically O(log n)

Interview Tip: Do not quote complexity without checking the exact variant and implementation.

Recursion, Backtracking & Dynamic Programming

Q61. Calculate factorial using recursion.

Solution: Base case: 0! = 1. For n>0, n! = n × (n–1)!.

Complexity: O(n) time, O(n) stack

```
function fact(n){
  if(n<=1)return 1;
  return n*fact(n-1);
}
```

Interview Tip: Define the state and recurrence before writing DP code. That makes the implementation much easier to verify.

Q62. Generate Fibonacci numbers efficiently.

Solution: Naive recursion repeats the same subproblems. Use iterative DP with two variables for O(n) time and O(1) space.

Complexity: O(n) time, O(1) space

```
function fib(n){
  let a=0,b=1;
  for(let i=0;i<n;i++) [a,b]=[b,a+b];
  return a;
}
```

Interview Tip: Define the state and recurrence before writing DP code. That makes the implementation much easier to verify.

Q63. What is dynamic programming?

Solution: DP stores solutions to overlapping subproblems and combines them through a recurrence. It can be top-down with memoization or bottom-up with tabulation.

Complexity: Varies by problem

Interview Tip: Define the state and recurrence before writing DP code. That makes the implementation much easier to verify.

Q64. 0/1 Knapsack: what is the DP state?

Solution: A common state dp[i][w] represents the maximum value using the first i items with capacity w. Transition chooses to exclude or include item i when it fits.

Complexity: O(nW) time, O(nW) space

Interview Tip: Define the state and recurrence before writing DP code. That makes the implementation much easier to verify.

Q65. What is the climbing stairs problem?

Solution: If one can climb 1 or 2 steps, ways(n)=ways(n–1)+ways(n–2), equivalent to Fibonacci-style DP.

Complexity: O(n) time, O(1) space

```
function climb(n){
  let a=1,b=1;
  for(let i=2;i<=n;i++) [a,b]=[b,a+b];
  return b;
}
```

Interview Tip: Define the state and recurrence before writing DP code. That makes the implementation much easier to verify.

Q66. Generate all subsets of an array.

Solution: Backtracking chooses for each element whether to include it or skip it.

Complexity: O(2^n) outputs, O(n) recursion depth

```
function subsets(a){
  const res=[];
  function bt(i,cur){
    if(i===a.length){res.push([...cur]);return;}
    bt(i+1,cur);
    cur.push(a[i]); bt(i+1,cur); cur.pop();
  }
  bt(0,[]); return res;
```

```
}
```

Interview Tip: Define the state and recurrence before writing DP code. That makes the implementation much easier to verify.

Q67. Generate all permutations.

Solution: At each recursion level, choose an unused element, append it, recurse, then undo the choice.

Complexity: O(n·n!) time to output permutations

```
function perms(a){
  const res=[];
  function bt(path,used){
    if(path.length===a.length){res.push([...path]);return;}
    for(let i=0;i<a.length;i++) if(!used[i]){
      used[i]=true; path.push(a[i]);
      bt(path,used); path.pop(); used[i]=false;
    }
  }
  bt([],[]); return res;
}
```

Interview Tip: Define the state and recurrence before writing DP code. That makes the implementation much easier to verify.

Q68. What is memoization?

Solution: Memoization caches the result of a function for each state so repeated calls return the stored result instead of recomputing.

Complexity: Usually reduces repeated recursion to polynomial/state complexity

Interview Tip: Define the state and recurrence before writing DP code. That makes the implementation much easier to verify.

Greedy & Problem-Solving Patterns

Q69. What is a greedy algorithm?

Solution: A greedy algorithm repeatedly chooses the best-looking local option. It is correct only when the problem has properties that make local choices globally optimal.

Interview Tip: Recognizing patterns is more valuable than memorizing isolated solutions. Ask: hashing, two pointers, sliding window, stack, binary search, BFS/DFS, or DP?

Q70. Activity selection problem.

Solution: Sort activities by finishing time and repeatedly choose the next activity whose start time is at least the finish time of the last selected activity.

Complexity: O(n log n) due to sorting

Interview Tip: Recognizing patterns is more valuable than memorizing isolated solutions. Ask: hashing, two pointers, sliding window, stack, binary search, BFS/DFS, or DP?

Q71. What is the two-pointer technique?

Solution: Maintain two indices that move through an array/string according to a condition. It often converts nested scans into a linear pass.

Interview Tip: Recognizing patterns is more valuable than memorizing isolated solutions. Ask: hashing, two pointers, sliding window, stack, binary search, BFS/DFS, or DP?

Q72. What is the sliding-window technique?

Solution: Maintain a contiguous window and expand/shrink its boundaries while preserving a condition. It is common for subarray/substring problems.

Complexity: Often O(n)

Interview Tip: Recognizing patterns is more valuable than memorizing isolated solutions. Ask: hashing, two pointers, sliding window, stack, binary search, BFS/DFS, or DP?

Q73. What are prefix sums?

Solution: A prefix-sum array stores cumulative totals, allowing a range sum to be computed as prefix[r]–prefix[l–1].

Complexity: Build O(n), range query O(1)

Interview Tip: Recognizing patterns is more valuable than memorizing isolated solutions. Ask: hashing, two pointers, sliding window, stack, binary search, BFS/DFS, or DP?

Q74. What is a monotonic stack?

Solution: A stack maintained in increasing or decreasing order. It is useful for next greater/smaller elements and range boundary problems.

Complexity: Often O(n) total

Interview Tip: Recognizing patterns is more valuable than memorizing isolated solutions. Ask: hashing, two pointers, sliding window, stack, binary search, BFS/DFS, or DP?

FOUNDATIONS

Complexity & Core CS Concepts

Q75. What is Big-O notation?

Solution: Big-O describes the growth rate of an algorithm's resource usage as input size grows. It ignores constants and lower-order terms for asymptotic comparison.

Interview Tip: In interviews, always state both time and space complexity when discussing an algorithm.

Q76. What is the difference between O(n) and O(log n)?

Solution: O(n) grows linearly with input size; O(log n) grows much more slowly. Binary search is a classic O(log n) example.

Interview Tip: In interviews, always state both time and space complexity when discussing an algorithm.

Q77. What is recursion stack space?

Solution: Each active recursive call consumes stack memory for parameters, local variables and return information. Maximum recursion depth determines the stack usage.

Interview Tip: In interviews, always state both time and space complexity when discussing an algorithm.

Q78. What is a hash collision?

Solution: A collision occurs when different keys map to the same hash bucket. Hash tables resolve collisions using techniques such as chaining or open addressing.

Interview Tip: In interviews, always state both time and space complexity when discussing an algorithm.

Q79. Array vs linked list?

Solution: Arrays provide O(1) indexed access and contiguous storage; linked lists provide node-based storage with O(n) indexed access and can support O(1) insertion after a known node.

Interview Tip: In interviews, always state both time and space complexity when discussing an algorithm.

Q80. Stack vs queue?

Solution: A stack is LIFO; a queue is FIFO. Choose based on the order in which items must be processed.

Interview Tip: In interviews, always state both time and space complexity when discussing an algorithm.

Q81. What is an adjacency list?

Solution: A graph representation storing a list of neighbors for each vertex. It is space-efficient for sparse graphs.

Interview Tip: In interviews, always state both time and space complexity when discussing an algorithm.

Q82. What is an adjacency matrix?

Solution: A V×V matrix where entry [i][j] represents an edge or weight. It gives O(1) edge lookup but uses $O(V^2)$ space.

Interview Tip: In interviews, always state both time and space complexity when discussing an algorithm.

ADVANCED TOPICS

OOP, Databases, APIs & System Design

Q83. What are the four pillars of OOP?

Solution: Encapsulation, abstraction, inheritance and polymorphism. Be able to give a small practical example for each.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

Q84. Composition vs inheritance?

Solution: Composition builds a class using contained objects and usually provides looser coupling. Inheritance models an is-a relationship. Prefer composition when inheritance does not express a genuine subtype.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

Q85. What is an API?

Solution: An API is a contract through which software components communicate. A REST API commonly uses HTTP methods and JSON representations.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

Q86. What is REST?

Solution: REST is an architectural style based around resources, stateless requests and standard HTTP semantics. Common methods include GET, POST, PUT/PATCH and DELETE.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

Q87. What is database normalization?

Solution: Normalization reduces redundant data and update anomalies by organizing related attributes into appropriate tables and relationships.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

Q88. What is an index in a database?

Solution: An index accelerates selected lookups but adds storage and write overhead. Index columns used frequently in filtering, joins or ordering after analyzing query patterns.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

Q89. What is caching?

Solution: Caching stores frequently needed data closer to the consumer so repeated requests can avoid expensive computation or database/network work.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

Q90. What is horizontal vs vertical scaling?

Solution: Vertical scaling increases resources on one machine. Horizontal scaling adds more machines/instances and often requires load balancing and distributed-state considerations.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

Q91. What is load balancing?

Solution: A load balancer distributes incoming traffic across multiple backend instances to improve availability, throughput and scalability.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

Q92. How would you design a URL shortener at a high level?

Solution: Create an API that maps a short identifier to a long URL, store the mapping in a database/cache, generate unique IDs, add redirects, and consider rate limits, expiration, analytics and abuse prevention.

Interview Tip: For design questions, clarify scale, availability, latency, consistency and security requirements before proposing components.

DATA STRUCTURES

Additional High-Frequency Coding Questions

Q93. How do you find the kth smallest element in an array?

Solution: For an unsorted array, options include sorting, a heap, or Quickselect. Quickselect gives average O(n) time; a heap is useful when k is small.

Complexity: Average O(n) with Quickselect; O(n log k) with a max-heap

Interview Tip: Before coding, identify the expected constraints. The constraints often tell you which complexity is acceptable.

Q94. How do you merge overlapping intervals?

Solution: Sort intervals by start time. Keep a current interval and merge the next interval when its start is no greater than the current end.

Complexity: O(n log n) time due to sorting, O(n) output space

Interview Tip: Before coding, identify the expected constraints. The constraints often tell you which complexity is acceptable.

Q95. How do you find the longest common prefix?

Solution: Compare characters column by column across all strings, or sort the strings and compare only the first and last strings.

Complexity: O(S) where S is the total relevant characters

Interview Tip: Before coding, identify the expected constraints. The constraints often tell you which complexity is acceptable.

Q96. How do you count character frequencies efficiently?

Solution: Use a hash map/object. Increment the count for every character as you scan the string.

Complexity: O(n) time, O(k) space

Interview Tip: Before coding, identify the expected constraints. The constraints often tell you which complexity is acceptable.

Q97. What is the difference between a min-heap and max-heap?

Solution: A min-heap keeps the smallest key at the root; a max-heap keeps the largest. Both maintain the heap property and support insertion/removal in O(log n).

Complexity: Peek O(1), insert/delete O(log n)

Interview Tip: Before coding, identify the expected constraints. The constraints often tell you which complexity is acceptable.

Q98. What is a trie?

Solution: A trie is a prefix tree for strings. Each path represents a prefix, making prefix queries efficient and independent of the number of stored strings in the basic complexity model.

Complexity: Insert/search O(L), where L is word length

Interview Tip: Before coding, identify the expected constraints. The constraints often tell you which complexity is acceptable.

Q99. How do you find the shortest path when edge weights can be negative?

Solution: Dijkstra is not valid with negative edges. Bellman-Ford can handle negative edge weights and can also detect reachable negative cycles.

Complexity: O(VE) time for Bellman-Ford

Interview Tip: Before coding, identify the expected constraints. The constraints often tell you which complexity is acceptable.

Q100. What is a union-find / disjoint-set data structure?

Solution: It maintains a collection of disjoint sets and supports find and union operations. Path compression and union by rank/size make operations nearly constant amortized time.

Complexity: Amortized $O(\alpha(n))$ per operation with optimizations

Interview Tip: Before coding, identify the expected constraints. The constraints often tell you which complexity is acceptable.

30-DAY CODING INTERVIEW PLAN

Days	Focus
1–4	Arrays, strings, hashing, frequency counting
5–7	Two pointers, sliding window, prefix sums
8–10	Linked lists, stacks and queues
11–14	Trees, BST, heap and recursion
15–18	Graphs, BFS, DFS and shortest paths
19–22	Sorting, binary search and greedy
23–26	Dynamic programming and backtracking
27–28	OOP, SQL, APIs and system design basics
29	Timed coding mock + review mistakes
30	Full mock interview + revision of weak topics

Final checklist: Solve without looking at the answer • explain aloud • test edge cases • state complexity • revisit wrong questions • practice under a timer • keep your code readable.